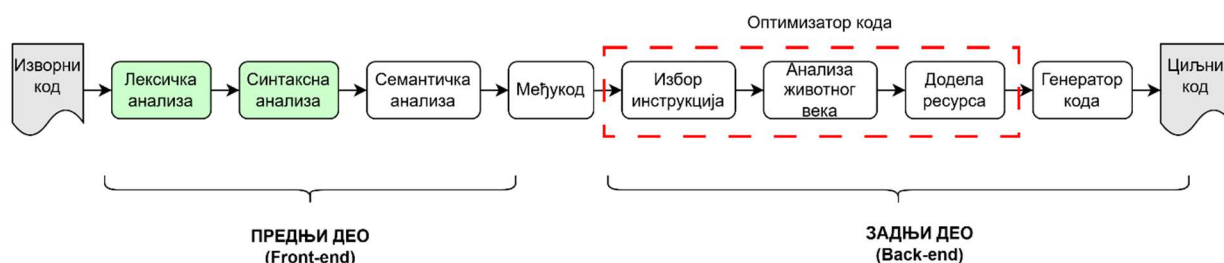


ДЕЛОВИ КОМПАЈЛЕРА ЛЕКСИЧКИ И СИНТАКСНИ АНАЛИЗАТОР

Компајлер (преводац) је програм који преводи изворни код написан на вишем програмском језику у код за циљну платформу. Основне фазе компајлера су:

- Лексичка анализа (препознавање симбола),
- Синтаксна анализа (провера граматичке исправности),
- Семантичка анализа (значење програма),
- Оптимизација,
- Генерисање кода за циљну платформу.



Слика 1: Основне фазе компајлера (преводиоца)

Лексички анализатор је део компајлера који треба да препозна све симболе (токене) у програму. **Синтаксни анализатор** проверава да ли су низови симбола добијени из лексичког анализатора у складу са граматиком језика, док **семантички анализатор** придружује значење синтаксно исправним изразима. У складу са тим, разликујемо синтаксне, лексичке и семантичке грешке.

Типови и примери грешака су дати у Табели 1:

Код	Тип грешке	Објашњење
<pre>if(a === b) c = a + b;</pre>	Лексичка грешка	Оператор === не постоји у језику (непознат симбол)
<pre>if(a == b) c = a + b</pre>	Синтаксна грешка	Недостаје ; (нарушена структура-синтакса изрази)
<pre>if (a = b) c = a - b;</pre>	Семантичка грешка	Додела (=) уместо поређења (==) доводи до погрешног значења

ЛЕКСИЧКИ АНАЛИЗАТОР

Лексички анализатор је део компајлера који:

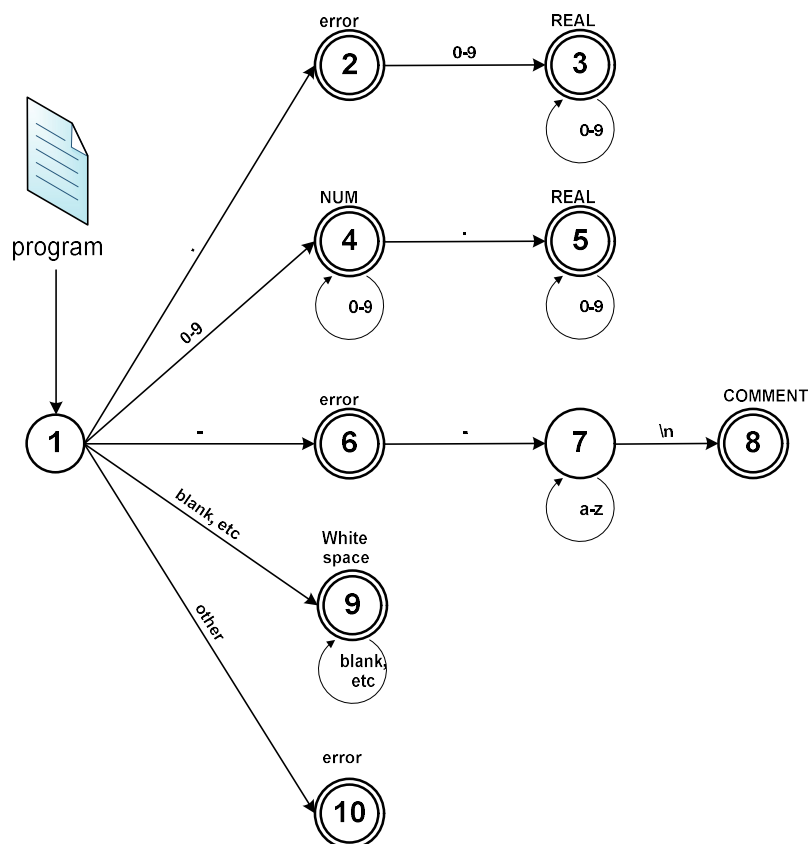
- чита улазну датотеку,
- препознаје токене,
- враћа листу токена.

Улаз	Изаз
<code>x = 10;</code>	ID(x) ASSIGN NUM(10) SEMICOLON

Лексички анализатор је реализован помоћу коначног детерминистичког аутомата који је формално дефинисан као скуп $D = (\Sigma, Q, q_0, F, \delta)$, где је:

- Σ – коначна азбука,
- Q – коначан скуп стања,
- q_0 – почетно стање,
- F – скуп прихватљивих (финалних) стања,
- δ – функција прелаза која одређује транзиције између стања.

Пример једног таквог аутомата дат је на следећој слици:



Слика 2: Пример једног коначног детерминистичког аутомата

Почетно стање је означено бројем 1. Стања у којима се завршава претрага називају се **финална стања** и означена су двоструким кругом (стања 2, 3, 4, 5, 6, 8, 9 и 10). Из датотеке се редом читају знакови и шаљу у коначни аутомат, одакле се, у зависности од знака, прелази у наредно стање. Критеријуми за прелазак између стања написани су на стрелицама које спајају стања. Тако, на пример, да би се препознао реалан број, први знак треба да буде тачка (.) или цифра (0-9). Уколико је први знак био тачка (.), из стања 1 прелази се у финално стање 2. Из финалног стања 2 је могуће: (1) прећи само у финално стање 3, уколико је наредни знак цифра (0-9), или (2) завршити рад коначног аутомата, уколико знак није цифра (0-9). У финалном стању 3 се очекују цифре (0-9). Када се појави знак који није из овог опсега, пријавиће се откривање симбола REAL (.12345, 1.2345, итд.).

За проналажење симбола могуће је користити следећа правила:

1. **Правило најдуже речи** – подразумева читање знакова из програма и прелазак у наредно стање аутомата све док се не пријави грешка лексичког анализатора.
2. **Правило приоритета** – подразумева читање знакова из програма и прелазак у наредно стање аутомата тако да се пријављује симбол чим аутомат пређе у финално стање.

Улаз	Правило најдуже речи	Правило приоритета
x+++y	(x++) + y	x + (++y)

Лексички анализатор који је приложен као материјал уз ову вежбу је реализован коришћењем правила најдуже речи и налази се у библиотеци *LexLib_d.lib*. Излаз из анализатора је листа токена прочитаних из улазне датотеке. Списак токена (у поставци задатка) се налази у *Token.h*.

СИНТАКСНИ АНАЛИЗАТОР

Синтаксни анализатор је део компајлера који проверава да ли су низови симбола добијени из лексичког анализатора у складу са граматиком језика. Он гради синтаксно стабло које представља структуру програма.

Дефиниција појмова

- **Речник:** скуп речи.
- **Реч:** коначни скуп симбола из коначне азбуке. (код програма)
- **Азбука:** симболи који се добијају као излаз из лексичког анализатора.
- **Продукција:** правило у граматичи којим се симбол може заменити низом других симбола
- **Граматика:** скуп продукција које описују језик са једним или више симбола са десне стране.

$symbol \rightarrow symbol\ symbol\ \dots\ symbol$

Сваки симбол може бити **терминални** или **нетерминални**:

1. **Терминални** симболи су они који су преузети из азбуке језика тј. лексичког анализатора (if, while, +, ...).
2. **Нетерминални** симболи су они који се налазе са леве стране исте продукције. Свака граматика поседује један нетерминални симбол који се означава као почетни симбол граматике. Извођење граматике се увек завршава терминалним симболима.

На следећој слици је дат пример једне граматике. Нетерминални симболи су: S, E и F, док су терминални: id, :=, -, ; и num.

$S \rightarrow id := E;$ $E \rightarrow F - E$ $F \rightarrow id$
 $E \rightarrow F$ $F \rightarrow num$

S
id := E;
id := F-E;
id := F-F;
id := id-num;

Слика 3: Граматика 1

Слика 4: Пример извођења за граматичу 1

Да би се сазнало да ли је нека реченица граматички исправна одређује се њено порекло – обавља се извођење. Извођење се обавља тако што се креће од почетног симбола, након чега се у итерацијама уместо нетерминалних симбола смењују њихове продукције.

Примери улазног програма који задовољавају пример граматике са Сlike 3:

Пример 1:	Пример 2:	Пример 3:
x := 10;	y := 1 - 3;	z := 1-4-y-x;

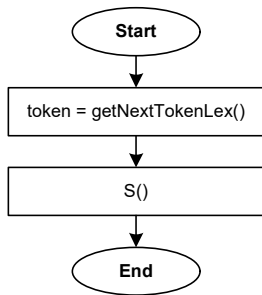
Алгоритам

Један од начина за реализацију синтаксног анализатора је коришћењем алгоритма са рекурзивним спуштањем. Свака продукција се претвара у реченицу позивима рекурзивних функција. Алгоритам са рекурзивним спуштањем поседује по једну **функцију за сваки нетерминални симбол** и по један **услов за сваку продукцију**, тако да ће за граматичу 1 бити потребне функције S, E и F у којима ће се испитивати постојање терминалних симбола id, :=, -, ; и num.

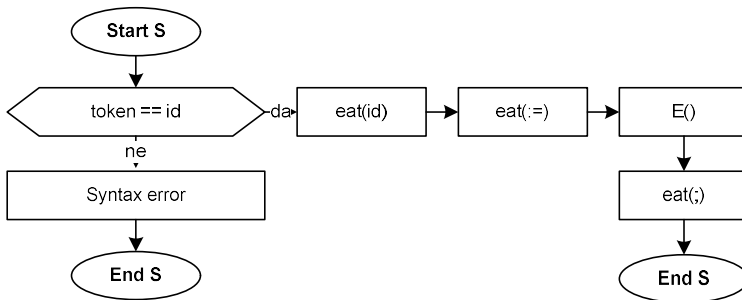
На почетку алгоритам добавља следећи (први) симбол и ставља га у глобалну променљиву **token**. Затим се позива функција S (Слика 5). У функцији S (Слика 5) се испитује да ли постоји правило у

граматици које почиње са добављеним симболом. Уколико не постоји овакво правило, пријављује се синтаксна грешка, док се у супротном ради извођење. Извођење се ради тако што се позива функција *eat* за све терминалне симболе и одговарајућа рекурзивна функција за нетерминалне симболе, у складу са граматиком. Функцији *eat* (Слика 9) се прослеђује симбол који се очекује да ће бити следећи у програму (*param*). Уколико се очекивани симбол (*param*) не поклапа са новооткривеним симболом, потребно је пријавити синтаксну грешку.

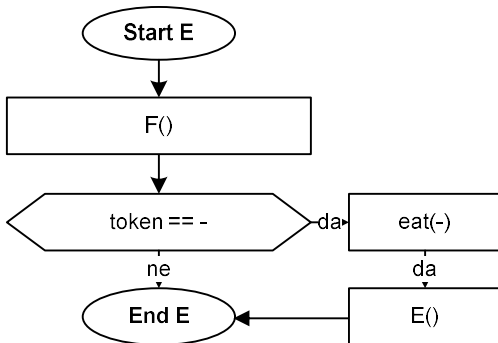
У функцијама *E* и *F* (Слика 7 и 8) се испитује да ли постоји правило у граматичи са тренутно актуелним симболом. Уколико постоји, обавља се извођење тог правила.



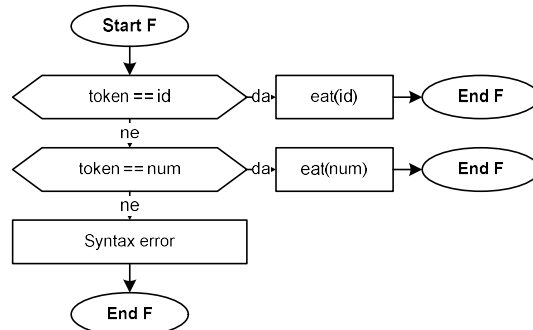
Слика 5: Синтаксни анализатор



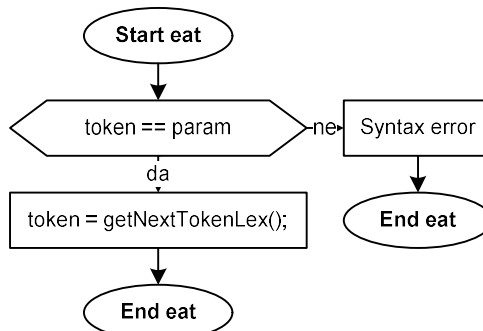
Слика 6: S функција



Слика 7: E функција



Слика 8: F функција



Слика 9: eat функција

РЕАЛИЗАЦИЈА СИНТАКСНОГ АНАЛИЗАТОРА

Лексички анализатор који је приложен као материјал уз ову вежбу је реализован коришћењем правила најдуже речи и налази се у библиотеци *LexLib_d.lib*. Излаз из лексичког анализатора је листа токена прочитаних из улазне датотеке.

На почетку рада програма је потребно позвати лексички анализатор да обави лексичку анализу. Ово се постиже позивом методе *Do()*. Уколико је повратна вредност методе истинита, значи да се лексичка анализа завршила без грешке. У супротном ће се исписати извештај о грешци. Излаз из лексичке анализе је листа токена учитаних из улазне датотеке.

```
typedef std::list<Token> TokenList;  
TokenList tokenList;
```

Ова листа не садржи *WHITE_SPACE* токене, јер они немају значај у датом примеру. Након лексичке анализе се позива синтаксна анализа. Приликом инстанцирања класе *SyntaxAnalysis* у конструктору се иницијализује итератор листе токена на почетак листе, након чега се позива метода *Do()* која означава извршење процеса синтаксне анализе.

```
TokenList::iterator tokenIterator;
```

Итератор служи за добављање наредног токена за анализу у синтаксном анализатору. То се постиже позивом методе:

```
Token getNextToken();
```

Типови подржаних токена су описани помоћу еnumerације *TokenType* у датотеци *Token.h*.

Приликом реализације задатка користити методу:

```
void eat(TokenType t);
```

која служи за проверу синтаксне исправности тренутног токена. Уколико тренутни токен није очекиваног типа (прослеђеног типа), метода исписује поруку о синтаксној грешци.

За испис токена у конзолу на располагању је метода класе *SyntaxAnalysis*:

```
void printTokenInfo(Token token);
```

Или, у случају грешке, метода:

```
void printSyntaxError(Token token);
```

Пример реализоване граматике:

У поставци задатка, у оквиру датотеке *SyntaxAnalysisExample.cpp* делимично је реализован синтаксни анализатор за граматика која је дефинисана у наставку.

$Q \rightarrow \text{begin } S L$	$S \rightarrow \text{if } E \text{ then } S \text{ else } S$	$L \rightarrow \text{end}$	$E \rightarrow \text{id} = \text{num}$
	$S \rightarrow \text{print } E$	$L \rightarrow S L$	

Сваки нетерминални симбол је реализован у оквиру истоимене функције у оквиру којих ће се испитати постојање терминалних симбола. За сваки терминални симбол потребно је проверити синтаксну исправност задатог токена позивом функције *eat()*.

ЗАДАЦИ

1. Навести бар 3 коначна симбола који су излаз из аутомата са Сlike 2.
2. Који су терминални, а који нетерминални симболи из граматике на Слици 10?
3. Употребом програмског језика C++ и већ постојећег лексичког анализатора, реализовати синтаксни анализатор са рекурзивним спуштањем. Користити граматiku 2 која је већ реализована у коду лексичког анализатора.

$Q \rightarrow \text{begin } L$	$L \rightarrow \text{end}$	$S \rightarrow \text{if } C \text{ then } K$	$E \rightarrow \text{id} = R$
	$L \rightarrow S L$	$S \rightarrow \text{if } C \text{ then } K \text{ else } K$	
		$S \rightarrow K$	$R \rightarrow \text{id}$
	$K \rightarrow \text{begin } L$		$R \rightarrow \text{num}$
	$K \rightarrow E;$	$C \rightarrow R == R$	$R \rightarrow \text{real}$

Слика 10: Граматика 2

4. Приликом успешног откривања симбола исписати његову текстуалну вредност у конзолу.
5. Уколико је улазни програм лексички и синтаксно исправан, исписати у конзолу поруку „*Syntax OK*“, у супротном исписати грешку код првог симбола који није синтаксно или лексички исправан.

ДОДАТНИ ЗАДАТАК

Осмислити програм „*program.txt*“ који има бар 2 угњеждене структуре који успешно пролази реализовану граматiku.